精确计算 CUDA Kernel 耗时

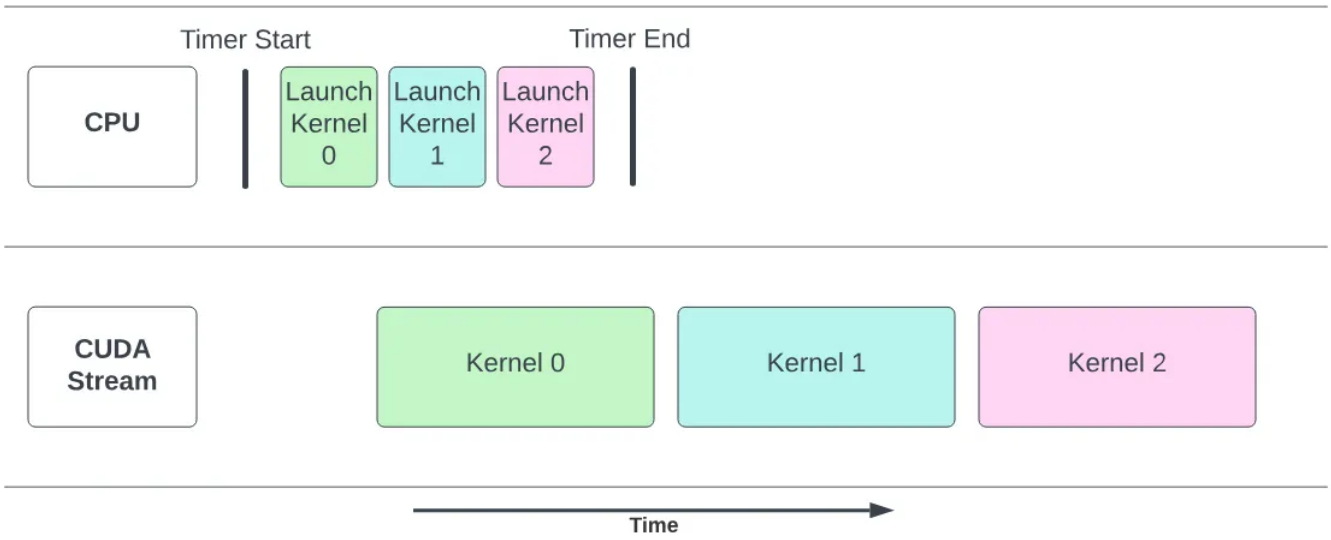
Introduction

为了能对模型推理过程中的 Kernel 耗时进行评估和优化，需要能精确且可重复地计算 CUDA Kernel 的耗时。但当涉及到 GPU 的异步任务时，做到这一点需要考虑一些技巧。

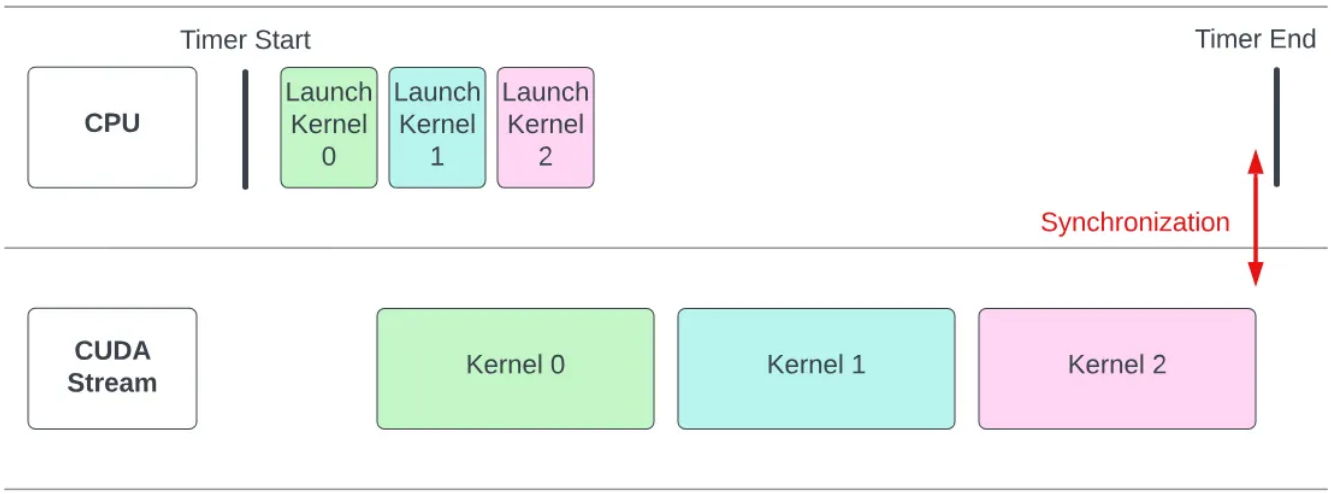
主机设备同步 (Host-Device Synchronization)

GPU 上的 Kernel 是**异步执行**的，即当 CUDA Kernel 在 GPU 上运行时，CPU 会继续将后续 Kernel 排队等待执行，这样可以避免诸如启动 Kernel 以及与 Python 解释器相关的开销成本造成的瓶颈。

但这也对 GPU 上操作的计时带来了困难，一些朴素的计时方法可能最终会对 **Kernel 启动**进行计时而不是 **Kernel 执行**，如下所示：



常见的解决方案是在进行时间测量之前调用 `torch.cuda.synchronize()`，这会等待所有 CUDA 流中的所有 Kernel 执行完成。



CUDA 事件 (CUDA Event)

然而，将显式同步与 `perf_counter` 结合使用时，我们不仅会计算 Kernel 执行时间，还会计算与 Kernel 启动相关的一些开销。此外，在分析性能关键型工作负载时，使用显式同步可能并不理想，因为这会导致速度下降。

CUDA 事件 (Event) 是一种巧妙的方法，可以**避免不必要的同步并隐藏 Kernel 启动开销**，例如：

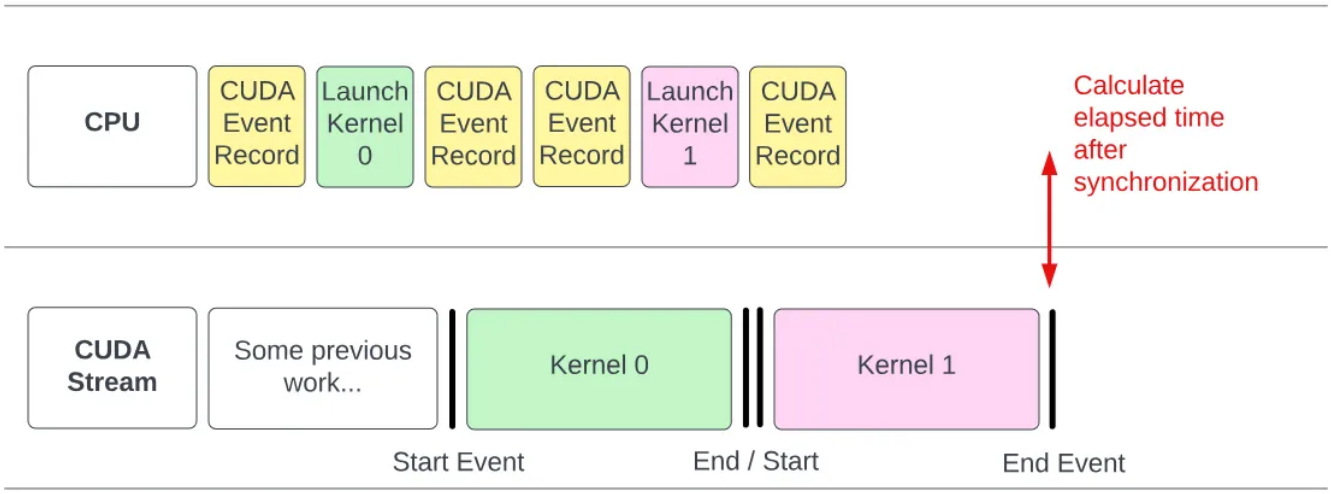
CUDA Event 计时 Python

```python
steps = 10
start_events = [torch.cuda.Event(enable_timing=True) for _ in range(steps)]
end_events = [torch.cuda.Event(enable_timing=True) for _ in range(steps)]

for i in range(steps):
    start_events[i].record()
    run_kernel()
    end_events[i].record()

torch.cuda.synchronize()
times = [s.elapsed_time(e) for s, e in zip(start_events, end_events)]
```

我们首先创建了两个 `torch.cuda.Event()` 对象列表， `record()` 方法本质上是在 Kernel 执行流中添加时间戳。我们在需要计时的操作之前和之后执行 record。在循环的末尾，我们必须在运行 `s.elapsed_time(e)` 之前包含一个 `synchronize()` 语句，如果省略则意味着 CPU 可能会在 GPU 完成所有工作之前尝试计算耗时，从而引发 `RuntimeError` 。下图展示了 `steps=2` 的 timeline：



预热步骤（Warm-Up Steps）

我们可以对上述示例进行进一步改进，即在计时运行之前加入预热步骤。这样做是为了**消除仅在训练或推理运行开始时产生的开销**。可能包括：

- 在遇到前几个输入张量后，PyTorch 的 JIT 融合器应用优化过程/代码生成。
- 在为给定的输入形状选择最佳卷积核时， `torch.cudnn.benchmark` 会进行动态微基准测试。
- 使用 `CUDA_MODULE_LOADING=LAZY` 和 `CUDA 11.7+` 将 Kernel 延迟加载到 CUDA 上下文中。
- PyTorch 的缓存分配器会调用 `cudaMalloc` 来初始扩展内存池，以便稍后重新使用，这会带来一定的开销。

举个例子：

Warmup Steps Python

```python
steps = 10

# Warmup steps
for _ in range(steps):
    run_kernel() # don't record time

start_events = [torch.cuda.Event(enable_timing=True) for _ in range(steps)]
end_events = [torch.cuda.Event(enable_timing=True) for _ in range(steps)]

for i in range(steps):
    start_events[i].record()
    run_kernel()
    end_events[i].record()

torch.cuda.synchronize()
times = [s.elapsed_time(e) for s, e in zip(start_events, end_events)]
```

固定时钟 (Fixed Clocks)

目前为止，我们主要关注提高性能分析结果的准确性。但如何才能确保结果的**可复现性**呢？GPU 时钟频率会因温度和功耗限制而发生显著变化。因此，固定时钟频率才能实现一致且可复现的基准测试。

以下是如何在 Python 中实现此功能的示例，采用了与 OpenAld 的 Triton 领域特定语言 (DSL) 和 Deepmind 的 AlphaTensor 存储库类似的方法。

固定时钟频率示例 Python

```python
import os
import subprocess

DEVICE = os.environ.get("CUDA_VISIBLE_DEVICES")
CLOCK_SPEED = 1350 # Must choose a clock speed that's supported on your device.

def set_clock_speed():
    """
    Set GPU clock speed to a specific value.
    This doesn't guarantee a fixed value due to throttling, but can help reduce variance.
    """
    process = subprocess.Popen("nvidia-smi", stdout=subprocess.PIPE, shell=True)
    stdout, _ = process.communicate()
    process = subprocess.run(f"sudo nvidia-smi -pm ENABLED -i {DEVICE}", shell=True)
    process = subprocess.run(f"sudo nvidia-smi -lgc {CLOCK_SPEED} -i {DEVICE}", shell=True)

def reset_clock_speed():
    """
    Reset GPU clock speed to default values.
    """
    subprocess.run(f"sudo nvidia-smi -pm ENABLED -i {DEVICE}", shell=True)
    subprocess.run(f"sudo nvidia-smi -rgc -i {DEVICE}", shell=True)
```

需要注意的是，使用 `nvidia-smi` 选择时钟频率**并不能保证** GPU 会以请求的频率运行。GPU 始终保留降低时钟频率（降频）以防止硬件损坏的能力。但是，通过将时钟频率设置为远低于最大值的值，我们可以确保降频的程度较小。

缓存清除 (Cache Flush)

另一个重要的考虑因素是确保在每次计时调用之间清除 GPU 内存缓存。这可以避免重复的 Kernel 执行利用缓存命中人为地降低延迟。一个简单的解决方案是每次运行传递不同的输入数据，但我们需要确保考虑周全。例如，在分析 `torch.nn.Linear` 时，仅仅更换输入数据可能不够，因为某些（静态）权重可能在多次运行后仍然保留在缓存中。

如果输入张量很大，频繁的重新生成操作也会减慢开发循环的速度。更稳健的解决方案是在每次迭代之间**显式地刷新缓存**。以下示例基于 Triton DSL 。其工作原理是写入足够的数据，以覆盖任何现有的缓存行，因为 Nvidia GPU 上的 L2 缓存采用回写策略 ，这意味着初始状态下 L2 缓存中会写入 `zeros` 数据。

清除 L2 缓存 Python

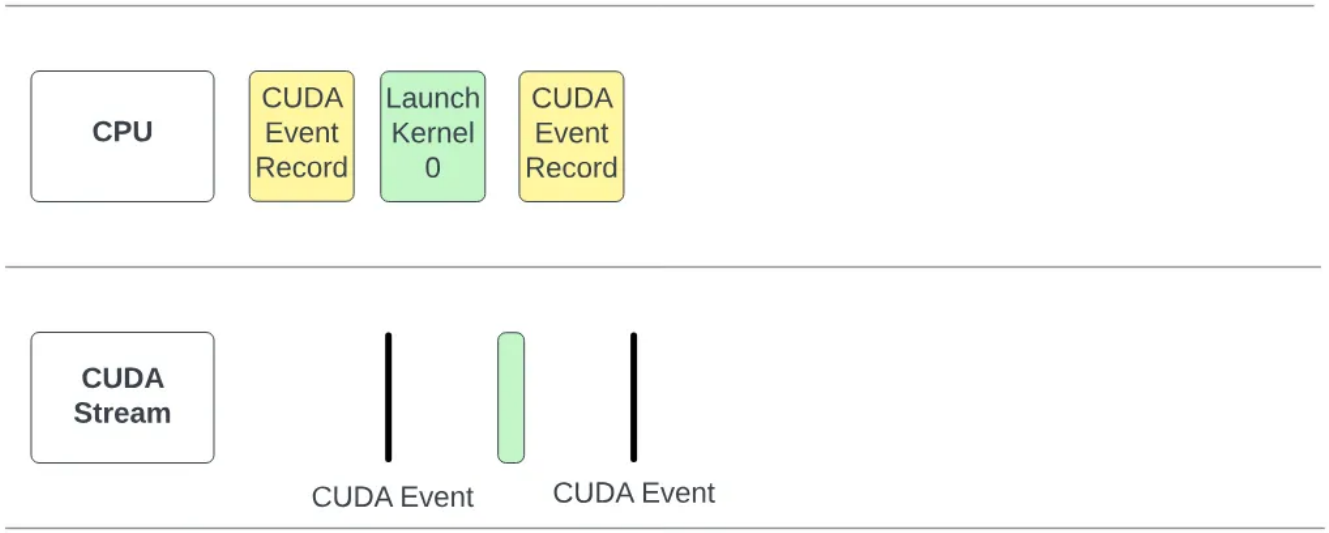
```python
# allocating 40MB to match L2 cache size on A100
x = torch.empty(int(40 * (1024 ** 2)), dtype=torch.int8, device='cuda')

def flush_cache():
    x.zero_()
```

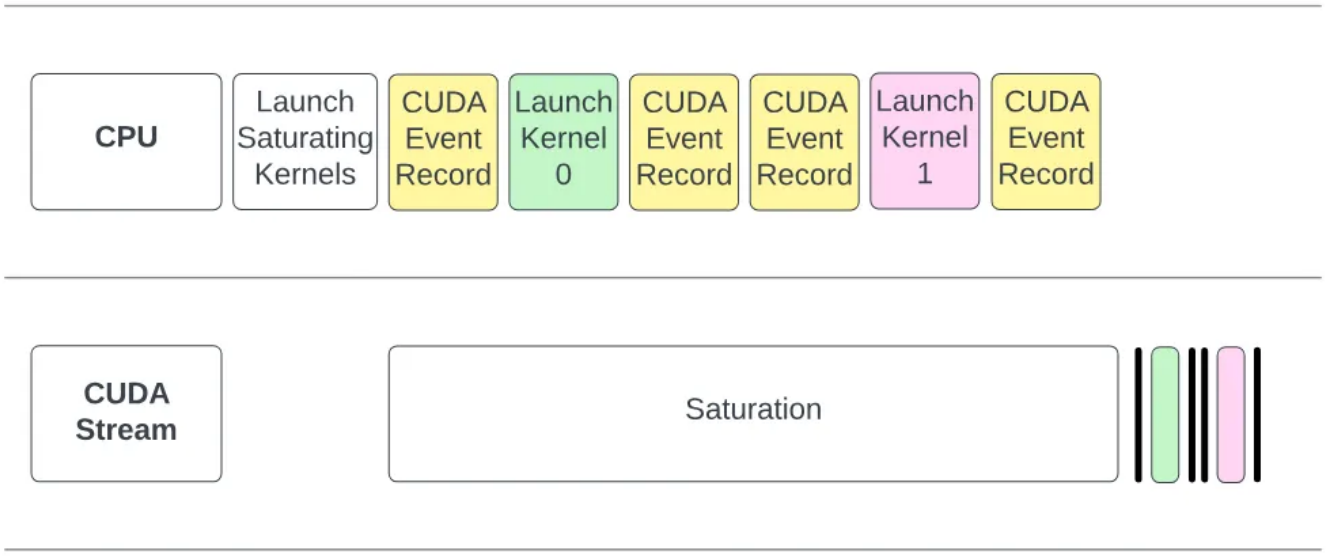
Sleep / CUDA Graphs

我们之前看到，CUDA 事件隐藏了启动 Kernel 的开销（主机启动 Kernel 到 Kernel 在 GPU 上执行之间的固定时间）。然而，这并非万全之策，因为**它假设目标 Kernel 与任务队列中相邻的 CUDA 事件之间没有时间间隔**。也就是说，它假设前一个 CUDA 事件在 Kernel 即将执行之前立即完成，而后一个 CUDA 事件在 Kernel 完成后立即启动。

当我们对执行速度快的轻量级 Kernel 进行计时时，这种假设可能不成立。Kernel 执行速度可能比 Kernel 启动速度更快，这意味着 GPU 的速度“超过”了 CPU。这会导致出现虚假结果，即 CUDA 事件增量中包含启动开销，如下图所示：



幸运的是，这个问题是有解决办法的。最简单的办法是**在启动目标 Kernel 之前先填满任务队列**。这样可以确保 Kernel 及其计时事件一起入队，而不是在下一个命令有机会进入队列之前就被执行：



我们应该如何实现这一点呢？一种简单的方法是**在执行我们感兴趣的操作之前启动一个开销足够大的 Kernel，从而造成积压**。更简洁的解决方案是让 GPU 等待固定数量的指令周期，可以使用 CUDA 的 `__nanosleep` 或 `torch.cuda._sleep()` 函数：

预先积压 Kernel Python

```python
set_clock_speed()

steps = 10

# Warmup steps
for _ in range(steps):
    run_kernel()

start_events = [torch.cuda.Event(enable_timing=True) for _ in range(steps)]
end_events = [torch.cuda.Event(enable_timing=True) for _ in range(steps)]

for i in range(steps):
    flush_cache()
    torch.cuda._sleep(1_000_000)

    start_events[i].record()
    run_kernel()
    end_events[i].record()

torch.cuda.synchronize()
times = [s.elapsed_time(e) for s, e in zip(start_events, end_events)]

reset_clock_speed()
```

第二种解决方案是**使用 CUDA Graph**。它通过将一系列独立的 Kernel 启动合并成一个 Kernel 来最大限度地减少启动开销。请注意，我们在图捕获过程中多次执行目标 Kernel，以分摊启动开销：

使用 CUDA Graph Python

```python
set_clock_speed()

steps = 10

# Warmup steps
for _ in range(steps):
    run_kernel()

start_events = [torch.cuda.Event(enable_timing=True) for _ in range(steps)]
end_events = [torch.cuda.Event(enable_timing=True) for _ in range(steps)]

# capture CUDA graph
graph = torch.cuda.CUDAGraph()
with torch.cuda.graph(graph):
    for _ in range(steps*10):
        run_kernel()

for i in range(steps):
    start_events[i].record()
    graph.replay()
    run_kernel()
    end_events[i].record()

torch.cuda.synchronize()
times = [s.elapsed_time(e) for s, e in zip(start_events, end_events)]

reset_clock_speed()
```

Reference

How to Accurately Time CUDA Kernels in Pytorch